Your organization was hacked by an outsider threat. It is believed that an unused port was used to access the system. Write a Python script that will scan any computer (or set of computers/servers) and produce an output that shows all open ports. Will be in Spyder 6

Absolutely! For your CIS 226 Programming for Cybersecurity assignment, we can build a **beginner-friendly Python Port Scanner** that you can run directly in **Spyder 6**.

The goal is to create a program that checks a computer or server for open ports and displays the results. This will help identify potentially unnecessary ports that could create security vulnerabilities.

1. Python Script – Network Port Scanner

We'll use Python's built-in `socket` library, so you don't need to install any additional packages.

Copy the following code into a new Python file in Spyder 6.

Network Port Scanner – Python Script



python

```python
# Network Port Scanner
# Cybersecurity Programming Assignment
# Purpose: Identify open ports on a computer or server.

import socket

# Scan a computer for open ports
def port_scanner(target, start_port, end_port):

    print("\n==================================")
    print("       NETWORK PORT SCANNER")
    print("==================================")

    print("Target:", target)
    print("Scanning ports", start_port, "through", end_port)
    print("----------------------------------")
```

```
    open_ports = []

    # Check each port in the selected range
    for port in range(start_port, end_port + 1):

        # Create a socket connection
        sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

        # Set connection timeout
        sock.settimeout(0.5)

        # Attempt to connect to the port
        result = sock.connect_ex((target, port))

        # Check if the port is open
        if result == 0:
            print("Port", port, "- OPEN")
            open_ports.append(port)

        # Close the connection
        sock.close()

    # Display scan results
    print("-----------------------------------")
    print("SCAN COMPLETE")

    if len(open_ports) > 0:
        print("Open ports found:", open_ports)
    else:
        print("No open ports found.")

    print("Total open ports:", len(open_ports))
    print("===================================")


# Main program
print("Welcome to the Network Port Scanner!")
```

```
target = input("Enter IP address or hostname to scan: ")

start_port = int(input("Enter starting port number: "))
end_port = int(input("Enter ending port number: "))

# Validate port range
if 1 <= start_port <= end_port <= 65535:

    port_scanner(target, start_port, end_port)

else:
    print("Invalid port range. Enter ports between 1 and 65535.")
```

2. How the Program Works

Here is what each important section does:

Python Code	Purpose
`import socket`	Allows Python to communicate with computers over a network.
`def port_scanner()`	Creates the function that performs the scan.
`socket.socket()`	Creates a network connection.
`sock.settimeout(0.5)`	Gives each connection attempt half a second.
`connect_ex()`	Attempts to connect to the selected port.
`if result == 0`	Determines whether the port is open.
`open_ports.append(port)`	Saves discovered open ports.
`sock.close()`	Closes the connection after checking the port.

3. Testing Your Program in Spyder 6

I recommend starting with your own computer.

Test 1: Scan your local computer

Enter:

- IP Address: `127.0.0.1`
- Starting Port: `1`
- Ending Port: `1000`

The IP address `127.0.0.1` refers to your own computer (localhost).

Test 2: Scan common ports

Try scanning ports 1–1024.

Some common ports include:

Port	Service	Purpose
21	FTP	File transfers
22	SSH	Secure remote access
23	Telnet	Remote access
25	SMTP	Email
53	DNS	Domain name resolution
80	HTTP	Web traffic
443	HTTPS	Secure web traffic
445	SMB	Windows file sharing
3389	RDP	Remote desktop

Remember that an open port isn't automatically a security vulnerability. It depends on the service running on that port and whether it needs to be accessible.

4. Example Output

Your output should look something like this:

```text
```

```
Welcome to the Network Port Scanner!

Enter IP address or hostname to scan: 127.0.0.1
Enter starting port number: 1
Enter ending port number: 1000

==================================
       NETWORK PORT SCANNER
==================================
Target: 127.0.0.1
Scanning ports 1 through 1000
----------------------------------
Port 135 - OPEN
Port 445 - OPEN
Port 5040 - OPEN
----------------------------------
SCAN COMPLETE
Open ports found: [135, 445, 5040]
Total open ports: 3
==================================
```

Your actual results will vary depending on which services are running on your computer.

5. Important Cybersecurity Considerations

The purpose of this program is to help an organization identify potentially exposed network services.

A security administrator can use the results to:

1. Identify unnecessary open ports.
2. Investigate unfamiliar services.
3. Disable services that are no longer needed.
4. Configure firewall rules to restrict access.
5. Reduce the organization's attack surface.

A couple of important limitations:

- This version scans one computer at a time.
- It checks TCP ports, not UDP ports.
- It only identifies whether a port accepts a TCP connection; it doesn't identify the application or verify whether the service is vulnerable.
- Only scan computers and servers you own or have permission to assess.

My recommendation for your assignment

I would start with this version because it demonstrates several important Python fundamentals:

- Variables
- Functions
- Loops
- Conditional statements
- Lists
- User input
- Exception-free basic socket communication

It is also manageable for someone learning Python for the first time.

One possible next improvement: We could upgrade this into a more advanced scanner that accepts multiple IP addresses, identifies common services running on open ports, and saves scan results to a `.txt` report.

This portion of the code comes up as 3 unidentified names: sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

That line is actually correct Python code for creating a network socket. In Spyder 6, the three unidentified names may be caused by the editor's code analyzer not recognizing the imported `socket` module properly.

Let's troubleshoot it.

1. Check your imports

At the very top of your script, make sure you have:

```python
import socket
```

2. Understanding the three names

In this line:

```python
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Each part has a purpose:

- `socket.socket` – Creates a network socket.

- `socket.AF_INET` – Specifies IPv4 addressing.
- `socket.SOCK_STREAM` – Specifies TCP communication.

3. Test the socket module independently

Create a new file in Spyder and run this:

```python
import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

print("Socket created successfully!")

sock.close()
```

If everything works, you should see:

```text
Socket created successfully!
```

4. If Spyder still identifies them as errors

Try these steps:

1. Go to **Consoles** → **Restart Kernel**.
2. Save your Python file.
3. Run the script again.

One question: Are the three unidentified names showing up as yellow or red underlines in the Spyder editor, or are you getting an actual error message when you run the program?

That distinction will help me identify whether this is just a Spyder warning or an actual Python problem.

Entering that line worked. In the port scan what are Port 135 and Port 445?

Good! That means your socket module is working correctly.

Ports **135 and 445** are both commonly used Windows networking ports. They are normal to see open on some Windows computers, but they are also worth understanding from a cybersecurity perspective.

Port 135 – Microsoft RPC

RPC (Remote Procedure Call)

- Allows Windows computers and applications to communicate with each other.
- Helps Windows services request information or perform tasks remotely.
- Commonly used for Windows administrative functions.
- Can be abused by attackers if vulnerable services are exposed.

Security concern: Attackers may exploit vulnerabilities in RPC services to gain unauthorized access or execute malicious commands.

Port 445 – Microsoft SMB

SMB (Server Message Block)

- Allows computers to share files and printers over a network.
- Used for accessing shared folders and network drives.
- Commonly used in Windows environments.
- Plays an important role in Windows file sharing.

Security concern: Port 445 is particularly important because attackers have historically exploited SMB vulnerabilities to spread malware and ransomware.

For example, the WannaCry ransomware attack in 2017 exploited a vulnerability in Microsoft's SMB implementation.

Quick Comparison

Port	Service	Primary Purpose	Security Risk
135	Microsoft RPC	Windows service communication	Remote exploitation
445	SMB	File and printer sharing	Malware and ransomware propagation

What does this mean for your assignment?

If your port scanner identifies ports 135 and 445 as open, it doesn't necessarily mean your computer has been compromised.

However, as a cybersecurity professional, you would want to investigate:

1. Is the service actually needed?
2. Is the computer exposed to the internet or only a private network?
3. Is Windows properly updated?
4. Are firewall rules restricting access to these ports?

An important takeaway: An open port isn't automatically a vulnerability. It's an entry point that needs to be evaluated and properly secured.

This is exactly what your port scanner is designed to help identify!